Setup and Credentials

> **Status**: Implementation reference for ADOR 0.5.1. Everything that must be installed, configured, or credentialed to run the Agentic Delivery Orchestrator (ADOR), on Windows 11 and Ubuntu 22+. Token scopes verified against the official GitLab and GitHub docs (July 2026); see References.

0. Setup Checklist (summary)

#	Item	Where	Notes
1	`git`	host	Any recent version.
2	Container runtime	host	Docker Desktop (Win) / Docker Engine or Podman (Ubuntu).
3	Python 3.11+ and `uv`	host	For ADOR itself and the locked package environment.
4	Git-platform CLI	host	`glab` (GitLab) and/or `gh` (GitHub), authenticated.
5	Executor CLI	container image	opencode / pi / codex / qwen.
6	Git-platform token	`.env` / CLI auth	GitLab `api` scope, or GitHub fine-grained (see below).
7	Model provider key	`.env` -> container	e.g. `ANTHROPIC_API_KEY`.
8	Git identity for the bot	config	`agent.name` -> commit author.
9	Branch protection	Git platform repo	Protect target branch; require approval so the bot cannot merge.
10	`config.toml`	ADOR repo	Non-secret settings (see Configuration_Reference.md).

1. Host Prerequisites

Same on both OSes (ADOR is portable Python; the heavy tooling lives in the container image):

- **git** on `PATH`.
- **Container runtime:** Docker Desktop (Windows 11) or Docker Engine / Podman (Ubuntu 22). ADOR runs the executor in a container.
- **Python 3.11+** plus **uv** for ADOR's locked runtime environment.
- **Git-platform CLI**, authenticated (section 2): `glab` and/or `gh`.
- **Network access** to the Git platform host (for example, a self-managed GitLab at `gitlab.example.com`) and to the model provider API.

2. Git Platform Credentials

ADOR needs to: read issues and their comments, post/edit issue comments (status + questions), read/apply labels, push a branch, create and update a (draft) merge or pull request, and read MR/PR review threads. It

must **not** be able to merge.

2.1 GitLab

Recommended token: a **Project Access Token** on the target project, or a **Personal Access Token** on a dedicated service/bot account if one exists. In a corporate instance where new users are hard to create, a PAT on an existing service account (or an approved human account) is acceptable - identity is marker-based, not account-based (see design doc 5c).

Scope: `api`.

- The `api` scope grants **complete read and write API access** for the token's scope, which covers everything ADOR does through the API: create/update merge requests, create and edit issue/MR **notes** (comments), read issues and discussions, and read/manage **labels**.
- For a **personal access token**, `api` **also** grants read/write to the repository over **Git-over-HTTP** (push/pull) - so a single `api` PAT covers both the API actions and the `git push` of the run branch.
- Do **not** add `sudo` or `admin_mode`.

Alternative (more granular): use a **fine-grained personal access token** to restrict the token to the single target project. Classic PATs have no per-endpoint scope for "merge requests only" - writing MRs/notes requires `api`.

Role / membership (this is what blocks merging):

- The token's account must be a **project member with the Developer role**. Developer can push to non-protected branches, create merge requests, and add notes/labels.
- Do **not** grant **Maintainer/Owner**. With the target branch protected and merges requiring approval, a Developer-level token **cannot merge** - which enforces the "human merges" boundary at the permission level.

Wire it up (CLI auth):

```
# self-managed host
glab auth login --hostname gitlab.example.com
# or non-interactive with a token:
export GITLAB_TOKEN=<token>        # ADOR / glab read this
```

If `glab` is already authenticated on the host (as in the current setup), ADOR can ride that auth and no token is needed in `.env`. Provide `GITLAB_TOKEN` only for direct API calls or to run `glab` inside the container.

2.2 GitHub (portability path)

Recommended token: a **fine-grained personal access token** scoped to the single target repository, with these repository permissions:

Permission	Level	Why
Metadata	Read	Mandatory baseline for fine-grained tokens.
Contents	Read and write	Clone/fetch and **push** the run branch.

Permission	Level	Why
Pull requests	Read and write	Create/update the draft PR; read review threads.
Issues	Read and write	Read issues, post/edit comments, manage labels.
Commit statuses	Read (optional)	Read CI status on the PR.

- Do **not** grant Administration. If the agent must edit files under `.github/workflows`, also add **Workflows: Read and write** (not needed for the current GitLab-hosted repo).
- **Classic PAT alternative:** the `repo` scope covers all of the above but is broader; prefer fine-grained.

Role: repository **collaborator with Write**. Branch protection with a required review prevents the token from merging (same boundary as GitLab).

Wire it up:

```
gh auth login                       # interactive
export GITHUB_TOKEN=<token>         # or non-interactive
```

3. Executor and Model Provider Credentials

The configured executor (**opencode, pi, codex, or qwen**) runs **inside the container** and needs a model-provider credential. This is the credential you almost certainly must supply. Select the executor with `[executor] default` in `config.toml`.

- **Which key** depends on the executor's configured provider, e.g. `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, or a gateway key. Put it in `.env`; ADOR injects only this subset into the container at `docker run` time.
- **opencode:** configured by `opencode.json` in the target repo (model, skills, permissions) plus the provider key in the environment. The repo's `opencode.json` already sets tool permissions and marks `git push` / `gh pr create` as `ask` (which safely blocks the executor from pushing).
- **pi** (`@earendil-works/pi-coding-agent`, config-selectable): same key pattern
  - a provider key in the environment (`--api-key` / env var). Pi reads `AGENTS.md` natively and gets its push/merge guardrails from the target repo's `.pi` **agentic-guard** extension (a return-value block enforced in every mode, including headless `--mode json`). **One caveat for unattended use:** Pi loads its project-local guard/skills only after the project is **trusted** (`ctx.isProjectTrusted()`), so the worktree must be pre-trusted for headless runs, or the guard does not load (container isolation still blocks remote writes regardless). Pinning the non-interactive trust step is part of the Pi live spike (00_Planning/TODO.md).
- **codex** (`@openai/codex`, config-selectable): runs through `codex exec` with `--json`, `--ignore-user-config`, `--disable plugins`, and a non-interactive sandbox/approval mode enforced by config validation. Codex reads `AGENTS.md` natively. When `stage_host_auth = true`, ADOR stages host `~/.codex/auth.json` into an isolated container `CODEX_HOME`.
- **qwen** (`@qwen-code/qwen-code`, config-selectable): runs through `qwen -p` with JSON output. Qwen reads `AGENTS.md` natively and relies on the target repo's `.qwen` hooks/deny list plus ADOR's container isolation. Qwen runs stateless per gate; config validation rejects yolo approval mode.
- **Never bake keys into the container image.** Pass them at launch only.

3.1 OAuth (ChatGPT account) vs API key - finding 2026-07-03

The current dev host's opencode auth is `type=oauth` (a ChatGPT-account login: rotating access + refresh tokens in `~/.local/share/opencode/auth.json`), **not an API key**. There is no `sk-...` string to put into `.env` - `OPENAI_API_KEY` only works with a real API key. The options:

1. **API key (required for production).** Create a key on platform.openai.com (API billing is separate from the ChatGPT subscription; for corporate use, a company OpenAI API account or internal gateway key). `OPENAI_API_KEY=sk-...` in `.env`; ADOR injects it into the container; opencode reads it from the environment. Per-token billing is what the `max_cost_usd` run budget bounds. Put this on the security sign-off list.
2. **Stage the host OAuth session (pilot-only).** Copy the host `auth.json` into the container at provision (the 2026-07-03 spike did this manually; now available as `[executor] stage_host_auth = true`). Caveats: the refresh token **rotates** - container and host sessions can invalidate each other, so it is fragile for anything long-running; and subscription accounts are intended for interactive use, so an unattended service on one is shaky both technically and terms-wise. Acceptable for crawl-phase tests on the dev machine only.
3. **A different provider with a plain API key** (e.g. `ANTHROPIC_API_KEY`, or a direct DeepSeek key) - same `.env` mechanism, different `model =` in `config.toml`. Note the "free" DeepSeek route *through opencode's gateway* also needs an opencode OAuth login, so it shares option 2's problem; a direct provider key does not.
4. **opencode's free gateway models** (verified available 2026-07-03: `opencode/nemotron-3-ultra-free`, `opencode/deepseek-v4-flash-free`, `opencode/mimo-v2.5-free`, `opencode/north-mini-code-free`). Needs a one-time `opencode auth login` for the opencode provider on the host; `stage_host_auth` then carries it into the container unchanged - and an opencode account avoids the ChatGPT subscription-terms concern entirely. Caveats: free-tier rate limits and weaker gate-work quality (pilots yes, real tickets to be assessed), and **data flow**: corporate code routes through opencode's gateway to the upstream provider on a free tier - needs the same security sign-off as any provider before non-throwaway use.

Decision pending: which option the pilot and production use. Until then, runs launched by ADOR need one of the above arranged manually.

4. Git Identity for the Bot

Not a secret, but required so commits are attributed consistently:

- `git config user.name` / `user.email` inside the container derive from `agent.name` (default `ADOR`). Use a recognizable identity, e.g. `ADOR <ador-bot@example.com>`.
- This is separate from the marker used to distinguish comments (design doc 5c); it only affects commit authorship.

5. Branch Protection (repo-side configuration)

This is what makes "the agent never merges" a hard rule rather than a convention:

- **Protect the target branch** (`develop` and/or `main`).
- **Require at least one approval** on the MR/PR before merge.

- Allow **Developer/Write** to push feature branches and open MRs, but restrict **merge** of the protected branch to Maintainer/approved reviewers.
- ADOR opens the MR/PR as a **draft**, which additionally cannot be merged until a human marks it ready.

6. Container Runtime and Image

- Install Docker/Podman (section 1).
- Build the **executor base image** once (`ador-executor-base:0.2.0`) with ADOR/executor tooling baked in: Python 3.11 + `uv`, Node 24 + `npm`, `git`, executor CLIs, and `glab`/`gh`.
- Build one **runtime-project image** per target project, extending the base image with that project's build/test tools. This repo includes the `ador-executor-cockpit-gcp:latest` example; other projects should use their own Dockerfile and tag.
- Create the persistent cache volumes referenced in `config.toml` (`ador-uv-cache`, `ador-npm-cache`) so `uv sync` / `npm ci` are fast.

7. Where Each Value Goes

Value	Location	Secret?
`GITLAB_TOKEN` / `GITHUB_TOKEN`	`.env` (or CLI auth store)	Yes
Model provider key (e.g. `ANTHROPIC_API_KEY`)	`.env` -> container	Yes
Git platform kind, repo path, labels, polling, budgets, etc.	`config.toml`	No
Bot name/slug, git identity	`config.toml`	No
Branch protection, approvals	Git platform project settings	n/a

- Secrets: `.env` (gitignored; template in `.env.example`), or on a server prefer systemd `EnvironmentFile=`, an OS keyring, or CI/CD variables.
- Non-secrets: `config.toml` (see `Configuration_Reference.md`).

8. Minimal vs Least-Privilege

- **Fastest to start:** rely on the already-authenticated `glab`/`gh` on the host for the Git platform, and supply only the model provider key in `.env`.
- **Least privilege (recommended for production):** a fine-grained, project-scoped Git-platform token on a dedicated service account with exactly the permissions in section 2, Developer/Write role only, and enforced branch protection.

References

- GitLab - Access token scopes (the `api`, `read_api`, `write_repository` table and the Git-over-HTTP footnote for personal access tokens): https://docs.gitlab.com/security/tokens/access_token_scopes/
- GitLab - Personal access tokens: https://docs.gitlab.com/user/profile/personal_access_tokens/
- GitLab - Fine-grained personal access tokens: https://docs.gitlab.com/auth/tokens/fine_grained_access_tokens/
- GitHub - Permissions required for fine-grained personal access tokens: https://docs.github.com/en/rest/authentication/permissions-required-for-fine-grained-personal-access-

tokens